EXPERIMENT NO. 6

Setting up Snyk in Jenkins

Name: Roll No.: Date: 16 October 2026

1. Aim

To integrate Snyk with Jenkins so that every build of a project scans its open-source dependencies for known vulnerabilities, and to fail the build when a finding at or above a chosen severity is reported. The project scanned is the CloudFire website (Next.js), repository `Donniedarko45/deploySourabh`.

2. Requirements

Component	Version used	Purpose
Jenkins	2.580.1 LTS (Docker image `jenkins/jenkins:lts-jdk21`, Java 21)	CI server that runs the scan
Snyk CLI	1.1308.1	performs `snyk test` / `snyk monitor`
Snyk Security plugin	plugin ID `snyk-security-scanner`	Snyk tool, credential type and pipeline step for Jenkins
Docker / Docker Compose	29.8.2 / v5.6.0	runs Jenkins in a container
Project	Next.js 16.1.0, React 19.2.3, `package-lock.json` v3	application under test
Snyk account	free plan	provides the API token and dashboard

3. Theory

Snyk is a developer security platform. **Snyk Open Source** is a software composition analysis (SCA) tool. It reads the manifest and lockfile (`package.json`, `package-lock.json`) and builds the full dependency graph, including transitive packages. Every package version is matched against the Snyk vulnerability database. For each issue Snyk reports the severity (low, medium, high, critical), the dependency path that introduced it, and the version that fixes it.

Jenkins is an open-source automation server. A *Pipeline* job reads its stages from a `Jenkinsfile` kept in the repository (pipeline as code). A *Freestyle* job runs build steps configured in the web UI.

DevSecOps / shift left. When the scan runs on every build, vulnerable dependencies are found before deployment. A *severity gate* makes the scan a quality gate: the build fails when a serious issue is found.

Ways to integrate Snyk with Jenkins

Method	How it works
1. Snyk Security plugin	Adds a *Snyk installation* tool (auto-downloads the CLI), a *Snyk API token* credential, the `snykSecurity` pipeline step / "Invoke Snyk Security task" build step, and a *Snyk Security Report* page on each build.
2. Snyk CLI in a build step	The Snyk CLI is installed on the Jenkins agent and run from an *Execute shell* step. The token is passed in the `SNYK_TOKEN` environment variable from a Jenkins secret.

Command	Meaning	Exit codes
`snyk test`	one-time test of the dependency graph	0 no issues · 1 issues found · 2 error · 3 no supported project
`snyk monitor`	uploads a dependency snapshot (package names and versions, not source code) to the Snyk dashboard for continuous alerts	0 on success
`--severity-threshold=high`	report only issues of this severity or higher	

4. Diagrams

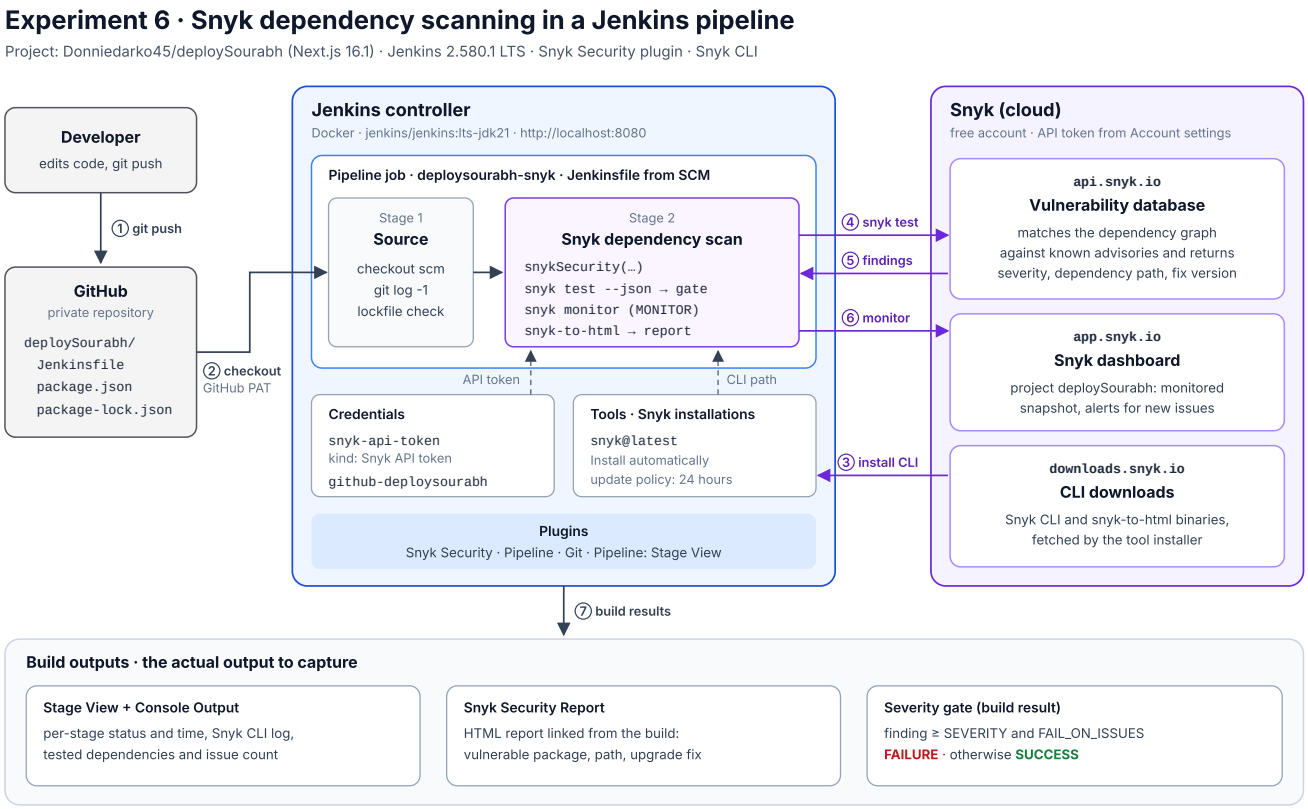


Figure 1. Architecture of Snyk scanning in a Jenkins pipeline. Numbers show the order of events in one build.

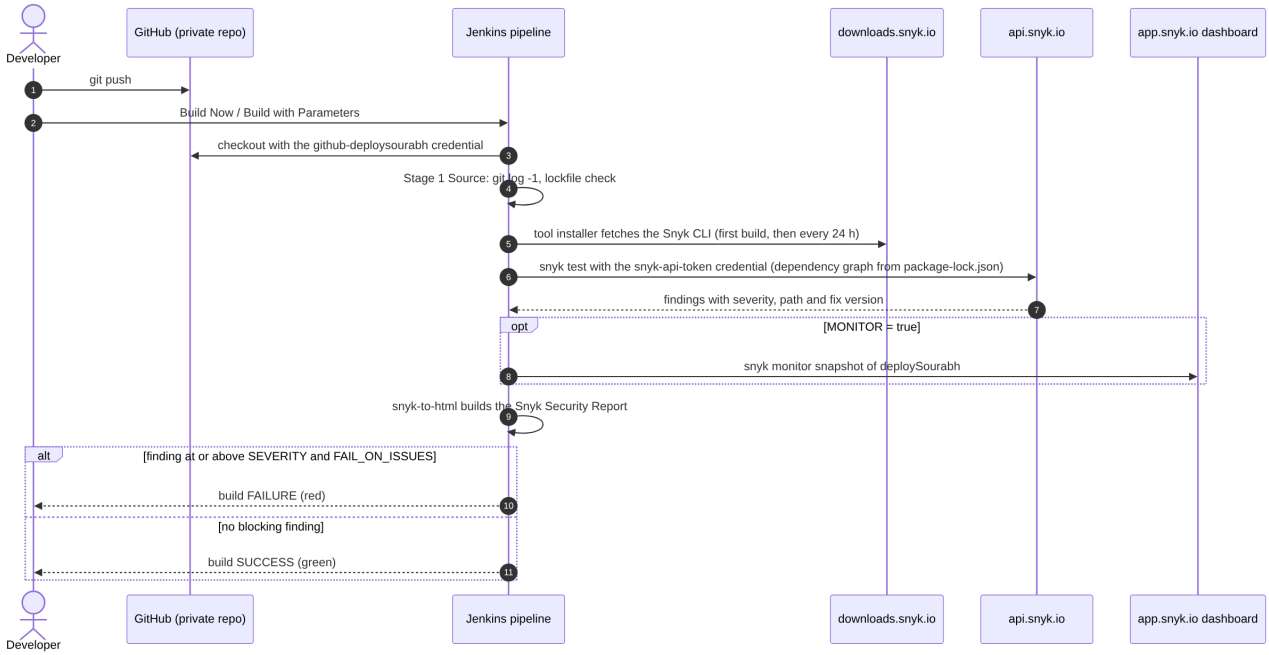


Figure 2. Sequence of one build with the Snyk Security plugin.

5. Procedure

Part A. Install and set up Jenkins (performed)

1. Build a Jenkins image that contains the Snyk CLI, then start it with the project mounted read-only at `/src` :

```
# Dockerfile
FROM jenkins/jenkins:lts-jdk21
COPY --from=snyk/snyk:node /usr/local/bin/snyk /usr/local/bin/snyk

$ docker build -t jenkins-snyk-demo:local .
$ docker run -d --name jenkins-snyk -p 8080:8080 \
    -v /home/user/deploySourabh:/src:ro jenkins-snyk-demo:local
$ docker exec jenkins-snyk cat /var/jenkins_home/secrets/initialAdminPassword
```

2. Open `http://localhost:8080` and paste the initial admin password on **Unlock Jenkins** (Figure 3).
3. On the plugin page choose **Skip Plugin Installations** (Figure 4), create the first admin user (Figure 5), keep the instance URL, then click **Save and Finish → Start using Jenkins** (Figures 6 and 7).

Part B. Get the Snyk API token

4. Sign in at `app.snyk.io` , open **Account settings → General → Auth Token**, reveal and copy the token. Never put the token in code or in the console log.

Part C. Method 2: Snyk CLI in a Freestyle job (performed)

5. **Dashboard → New Item**, name `deploysourabh-snyk` , type **Freestyle project**, OK.
6. **Build Steps → Add build step → Execute shell**, enter the script in Section 6.2 and click **Save** (Figure 8).
7. With a real token, add it as a *Secret text* credential and bind it to `SNYK_TOKEN` using **Build Environment → Use secret text(s) or file(s)** (Credentials Binding plugin).
8. Click **Build Now** and open **Console Output** (Figures 9 and 10).

Part D. Method 1: Snyk Security plugin with a Jenkinsfile

9. **Manage Jenkins → Plugins → Available plugins**, search **Snyk Security**, install and restart. (The `jenkins/` folder in the repository builds an image with this plugin preinstalled: `cd jenkins && docker compose up -d --build` .)
10. **Manage Jenkins → Credentials → System → Global credentials → Add Credentials**: Kind **Snyk API token**, paste the token, ID `snyk-api-token` . For the private repository also add a **Username with password** credential (GitHub user + read-only personal access token) with ID `github-deploysourabh` .
11. **Manage Jenkins → Tools → Snyk installations → Add Snyk**: Name `snyk@latest` , tick **Install automatically**, version `latest` , update policy 24 hours. Save.
12. **New Item → Pipeline**. Definition **Pipeline script from SCM**, SCM **Git**, URL `https://github.com/Donniedarko45/deploySourabh.git` , credential `github-deploysourabh` , Script Path `Jenkinsfile` . Save and click **Build Now**.
13. Check the Stage View, the Console Output, the **Snyk Security Report** link on the build, and the project on the Snyk dashboard.

6. Program

6.1 Jenkinsfile (Method 1, committed in the repository)

```
pipeline {
    agent any

    options {
        buildDiscarder(logRotator(numToKeepStr: '20'))
        disableConcurrentBuilds()
        timeout(time: 20, unit: 'MINUTES')
    }

    parameters {
        choice(name: 'SEVERITY', choices: ['high', 'critical', 'medium', 'low'],
               description: 'Lowest severity that counts as a finding')
        booleanParam(name: 'FAIL_ON_ISSUES', defaultValue: true,
                     description: 'Fail the build when Snyk reports a finding at or above SEVERITY')
        booleanParam(name: 'MONITOR', defaultValue: true,
                     description: 'Send a dependency snapshot to the Snyk dashboard (snyk monitor)')
    }

    stages {
        stage('Source') {
            steps {
                sh '''
                    git log -1 --format='Commit %h on %cd: %s' --date=iso
                    test -f package-lock.json
                    echo "Lockfile present: package-lock.json"
                '''
            }
        }

        stage('Snyk dependency scan') {
            steps {
                snykSecurity(
                    snykInstallation: 'snyk@latest',
                    snykTokenId: 'snyk-api-token',
                    targetFile: 'package.json',
                    projectName: 'deploySourabh',
                    severity: params.SEVERITY,
                    failOnIssues: params.FAIL_ON_ISSUES,
                    failOnError: true,
                    monitorProjectOnBuild: params.MONITOR
                )
            }
        }
    }

    post {
        success {
            echo "Snyk gate passed: no ${params.SEVERITY}+ findings blocked this build."
        }
        failure {
            echo 'Snyk gate failed. Open "Snyk Security Report" on this build for the findings and
fixes.'
        }
    }
}
```

6.2 Execute shell build step (Method 2, used for the lab run)

```
echo "=== Stage 1: Source ==="
cp /src/package.json /src/package-lock.json .
git -c safe.directory=/src -C /src log -1 --format='Commit %h on %cd: %s' --date=iso
test -f package-lock.json && echo "Lockfile present: package-lock.json"

echo "=== Stage 2: Snyk dependency scan ==="
snyk --version
getent hosts api.snyk.io || echo "api.snyk.io: DNS resolution failed from this network"
snyk test --file=package.json --severity-threshold=high
```

7. Output

> The lab was carried out on 16 October 2026 in a Linux cloud sandbox with Docker. Its network allowed Docker Hub and the npm registry, but blocked the Jenkins update center (`updates.jenkins.io`) and all Snyk servers (`*.snyk.io`). All screenshots and logs below come from that run; screenshots are only trimmed of empty space.

7.1 Jenkins installation and setup

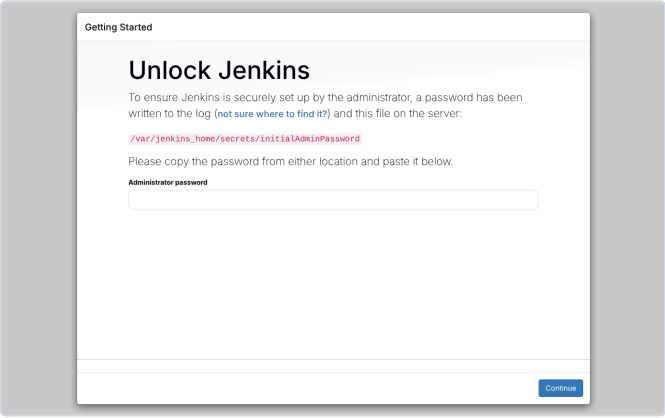


Figure 3. Unlock Jenkins

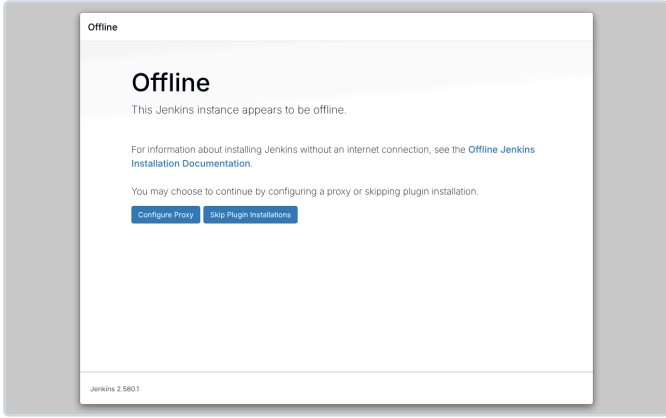


Figure 4. Plugin step: instance offline, plugins skipped

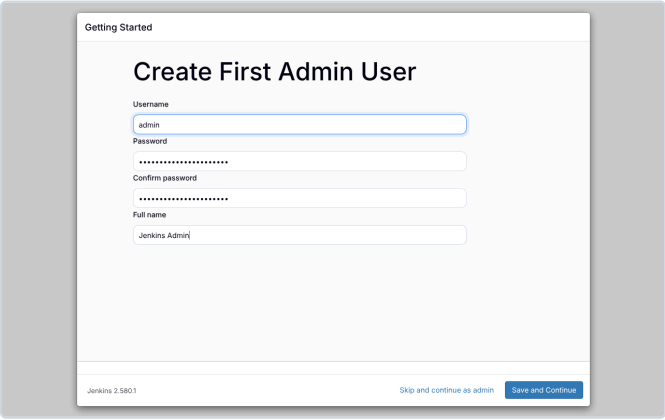


Figure 5. Create First Admin User

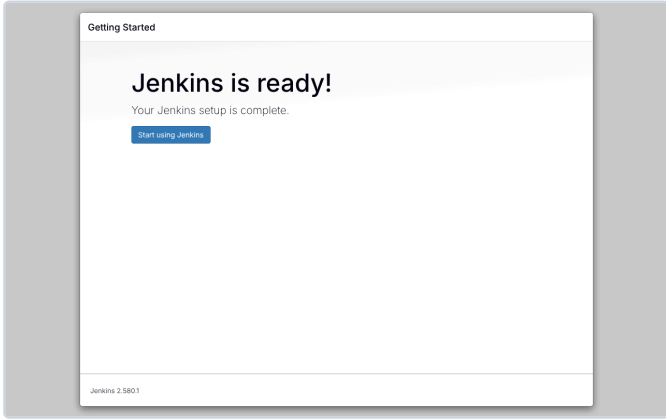


Figure 6. Jenkins is ready (2.580.1)

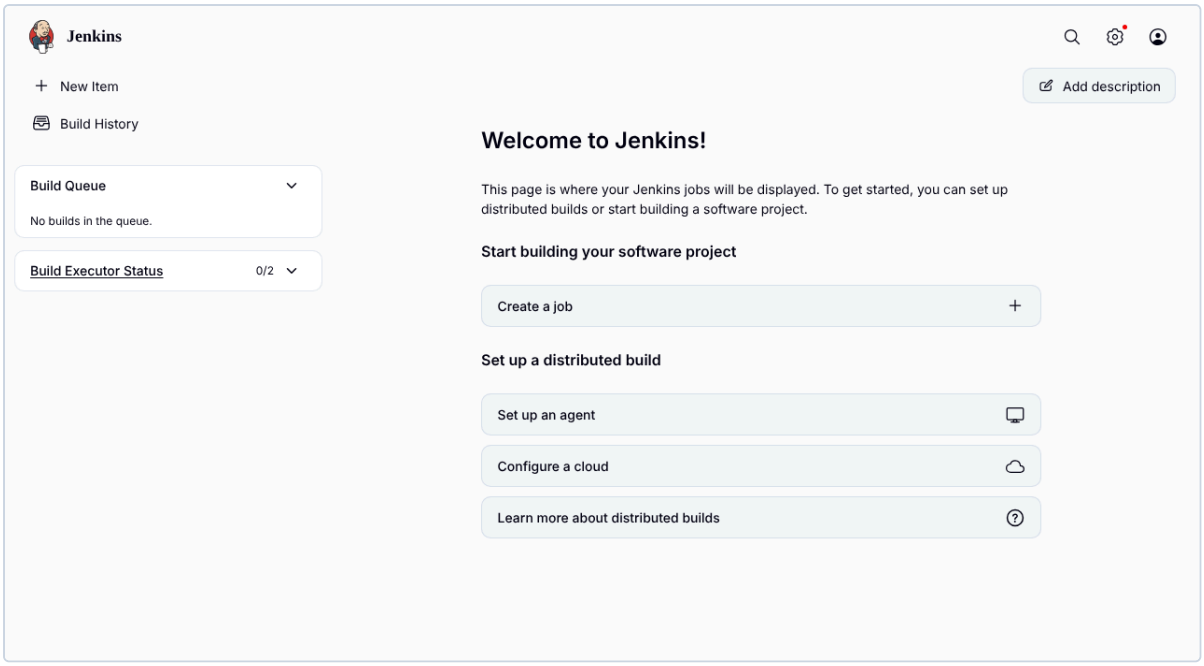


Figure 7. Jenkins dashboard after setup

7.2 Job configuration

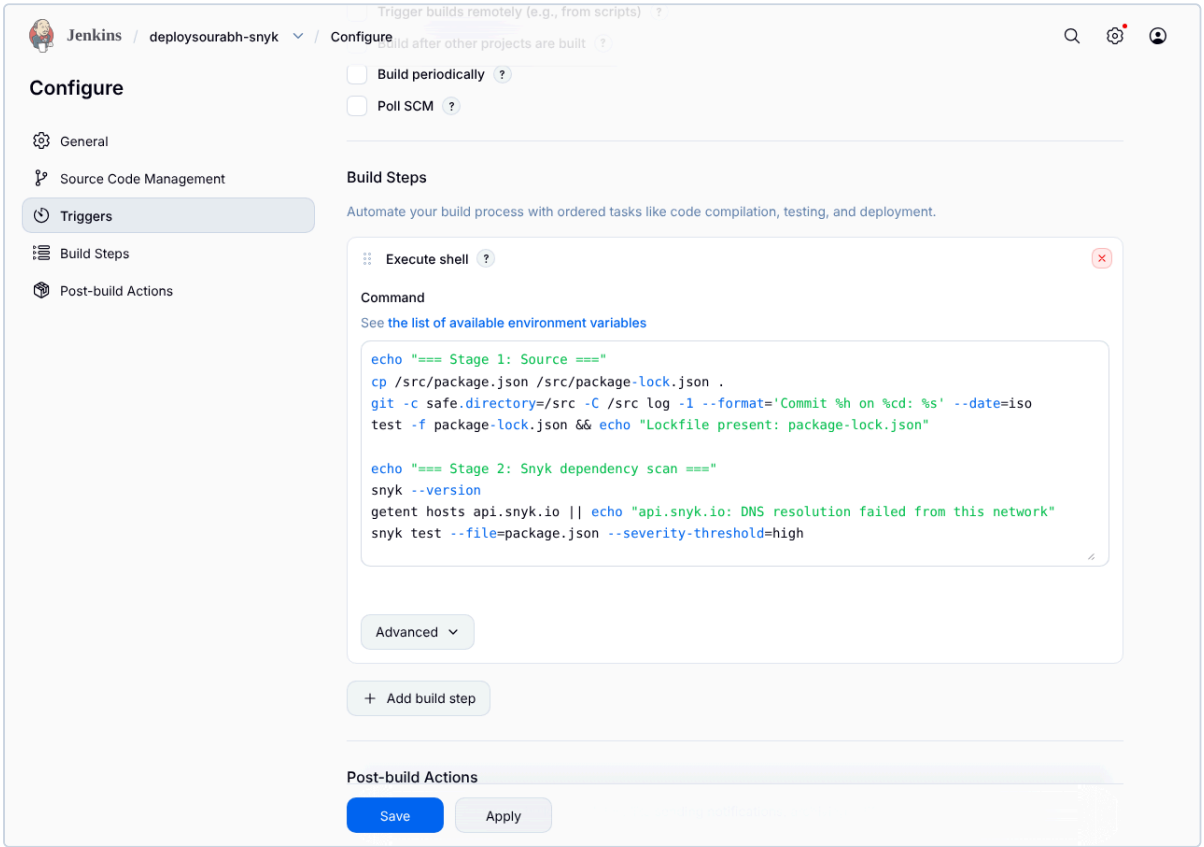


Figure 8. Job `deploysourabh-snyk`: Execute shell step running the Snyk CLI

7.3 Build #1

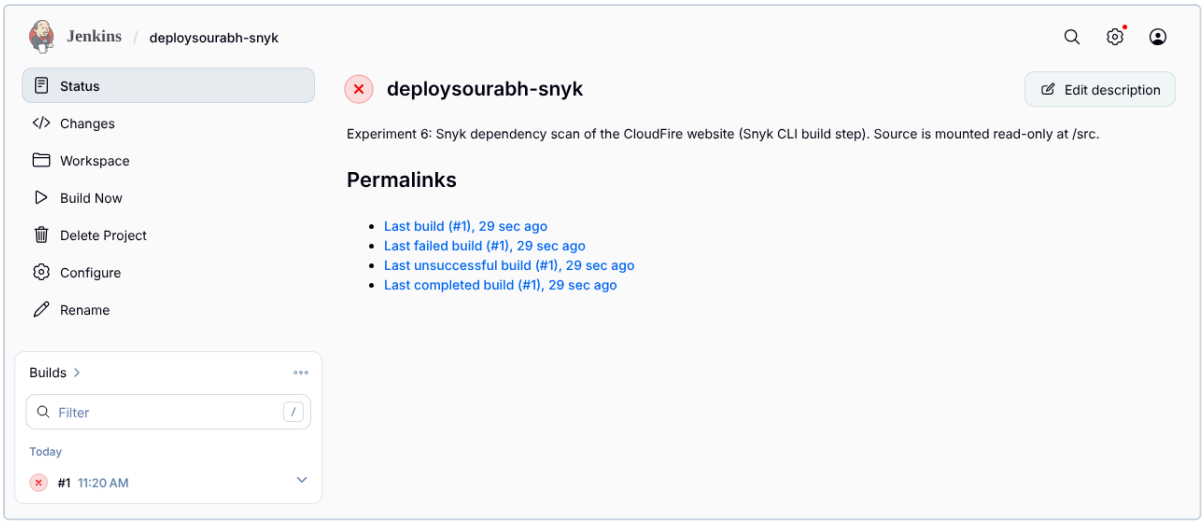


Figure 9. Job page: build #1 failed

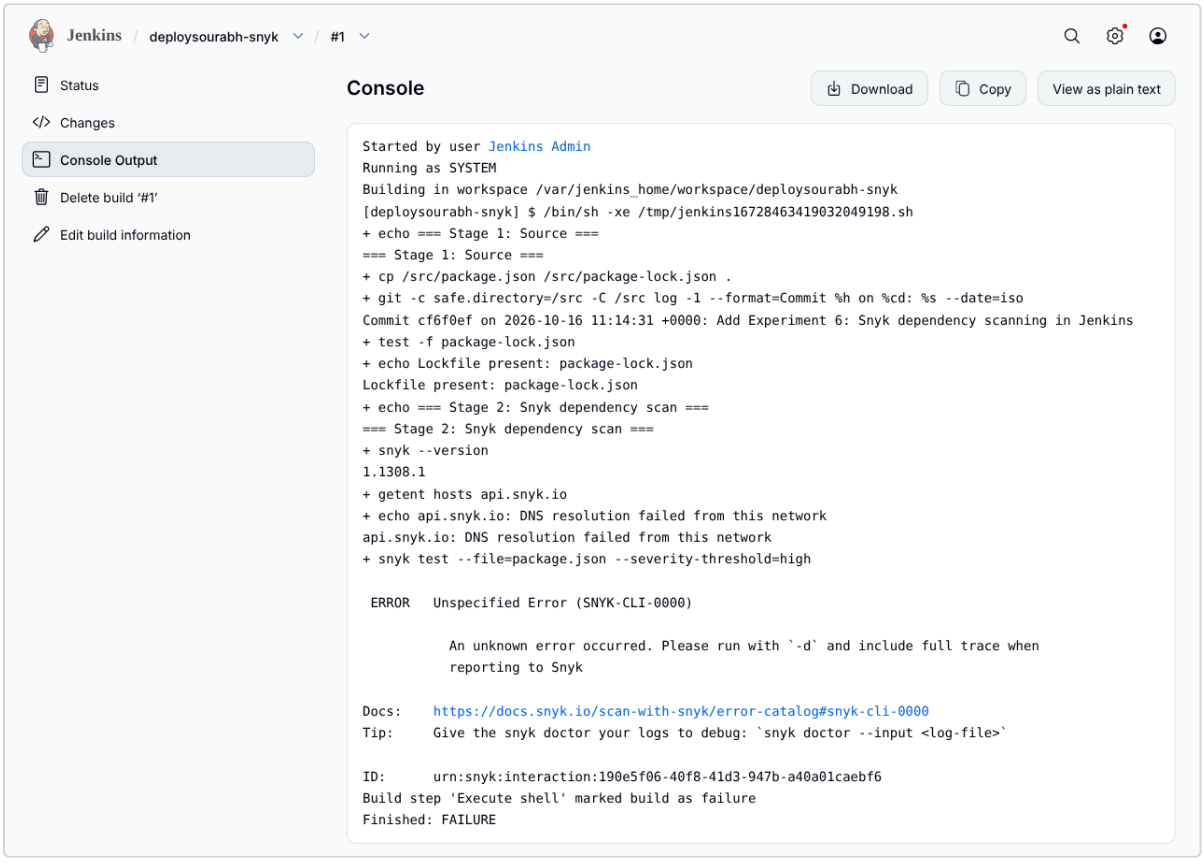


Figure 10. Console Output of build #1

Build record returned by the Jenkins REST API (`scripts/collect-jenkins-snyk-output.sh`):


```
{"_class":"hudson.model.FreeStyleBuild","duration":4044,"number":1,"result":"FAILURE",
 "timestamp":1792149659215,"url":"http://127.0.0.1:8080/job/deploysourabh-snyk/1/"}
```

Cause of the Snyk error, from `snyk test -d` (debug log) in the same environment; `snyk test` exited with code 2 (error):

```
GET https://api.snyk.io/rest/self?version=2024-04-22
< error: DNS resolution failed
Failed to determine default value for "ORGANIZATION":unable to retrieve org ID:
Get "https://api.snyk.io/rest/self?version=2024-04-22": DNS resolution failed
```

7.4 Snyk Security plugin installation

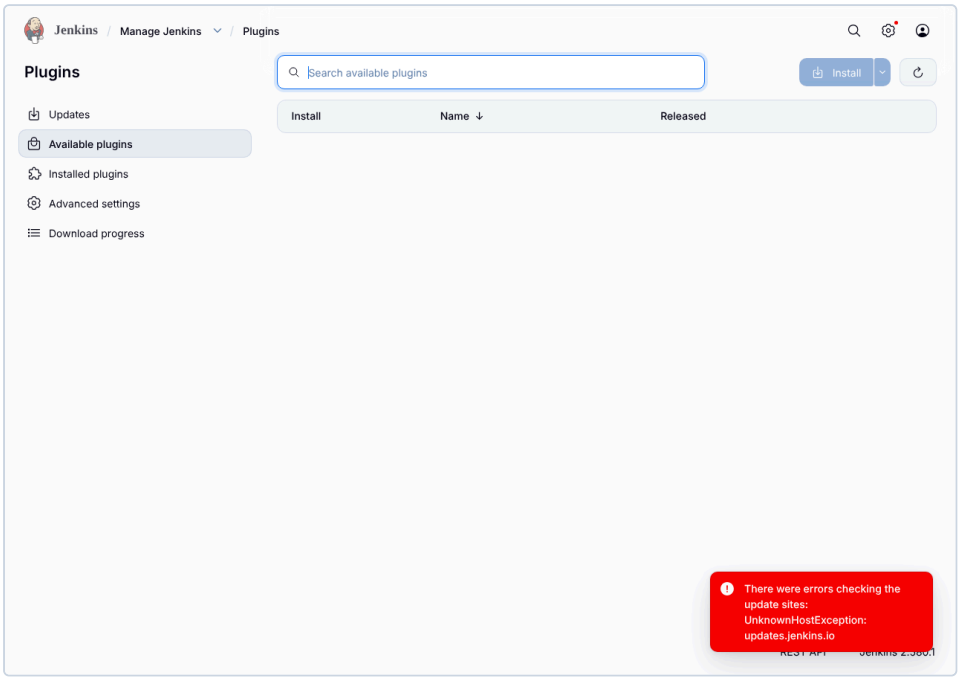


Figure 11. Manage Jenkins → Plugins: "UnknownHostException: updates.jenkins.io"

Building the plugin image in `jenkins/` stopped at the same point:

```
$ docker compose build
Unable to retrieve JSON from https://updates.jenkins.io/update-center.json?version=2.580.1:
updates.jenkins.io: Name or service not known
Error getting update center json
```

7.5 Tool and project checks

```
$ docker run --rm --entrypoint java jenkins/jenkins:lts-jdk21 -jar /usr/share/jenkins/jenkins.war
--version
2.580.1

$ docker run --rm --entrypoint snyk snyk/snyk:node --version
1.1308.1

$ npm ls --package-lock-only --depth=0          # lockfile in sync with package.json (exit code 0)
cloudfireiitservice@0.1.0 /home/user/deploySourabh
+-- next@16.1.0
+-- react-dom@19.2.3
+-- react@19.2.3
…
```

7.6 Cross-check: npm audit (not Snyk)

`npm audit --package-lock-only --omit=dev` (GitHub advisory database, reachable from the lab network) shows what the Snyk gate is expected to catch:

Severity	Package	Affected range	Fix
critical	next (direct)	9.3.4-canary.0 – 16.3.7	next@16.4.2
high	postcss	≤ 8.5.22	next@16.4.2
high	sharp	≤ 0.35.5-rc.1	next@16.4.2
high	nanoid	≤ 3.3.17	`npm audit fix`
high	source-map-js	1.0.0 – 1.2.1	`npm audit fix`
moderate	baseline-browser-mapping	≥ 2.0.0 < 2.11.0	`npm audit fix`
moderate	fflate	0.8.0 – 0.8.2	`npm audit fix`

Total: 7 vulnerabilities (1 critical, 4 high, 2 moderate).

8. Observations

1. Jenkins 2.580.1 started in Docker and the setup wizard completed. The wizard reported the instance as offline because the update center was unreachable.
2. Job `deploysourabh-snyk` ran both stages. Stage 1 read commit `cf6f0ef` and found the lockfile. Stage 2 ran Snyk CLI 1.1308.1.
3. `snyk test` could not resolve `api.snyk.io` and exited with code 2. Jenkins marked build #1 **FAILURE**. The pipeline fails closed: a build whose dependencies could not be scanned does not pass.
4. The Snyk Security plugin could not be installed in this environment (Figure 11). The `Jenkinsfile` for Method 1 is ready in the repository.
5. npm audit reports a critical issue in `next@16.1.0`, so the Snyk gate (`high`) is expected to fail until `next` is upgraded to 16.4.2 or later.

9. Result

Jenkins 2.580.1 LTS was installed and configured, and a Jenkins job integrating the Snyk CLI 1.1308.1 was created and executed against the CloudFire website (build #1). The integration ran as designed up to the Snyk API call. The build **failed** because the lab network could not reach `api.snyk.io`, which shows that the build fails safely when the scan cannot run. A Snyk Security plugin pipeline with a high-severity gate was also written. A Snyk vulnerability report needs internet access to Snyk and a Snyk API token.

10. Precautions

- Store the Snyk token only in Jenkins credentials; never in the Jenkinsfile, console log or repository.
- Keep `package-lock.json` committed and in sync with `package.json`; Snyk stops on an out-of-sync lockfile.
- `snyk monitor` uploads package names and versions, not source code; set `MONITOR=false` to skip it.